

UCLA

and Large Language Models for Code Intelligence

CS 239: Topics in Computer Science: Programming Languages and System

Website: https://ucla-coin-lab.github.io/teaching/cs239_fall2026/

Instructor:

Robin Ding

UCLA Computer Science





- **Instructor:** **Robin Ding, Assistant Professor, Computer Science**
 - **PI:** UCLA Code Intelligence (COIN) Lab
 - **Research:** LLMs and Agents for Code Intelligence
 - **Office:** Engineering VI, Office 295B
 - **Email:** yrbding@cs.ucla.edu (All email subjects start with “[CS239]”)
 - **Website:** yrbding.github.io

- **Content**

- **Seminar:** Paper Presentation and Discussion
 - Fundamental papers of modern agentic LLMs and code intelligence
 - Model architecture, data, pre-/post-training, inference & agents
 - Software Engineering, Cybersecurity, Formal Verification
- **Recommended Prerequisites:** Advanced knowledge in artificial intelligence (with a focus on large language models) and software engineering.
 - CS 161: Fundamentals of Artificial Intelligence, or the equivalent.
 - CS C160F: Foundation Models: Principles and Practice, or the equivalent.
 - CS 130: Software Engineering, or the equivalent.

- **Course Content**

- **Paper Presentation and Discussion**

- JUST one presentation for you: 35-min presentation + 15-min QA
 - Register the paper by **Oct 5!!**
 - Read other papers and actively discuss

- **Course Project:** Any topic related to “LLMs and code intelligence”

- Team work: min of **3**, max of **5 (due: Oct 9)**
 - Three milestones: Proposal (Oct 14, one-page report due Oct 21), mid-term (Nov 4&9, two-page report due Nov 11), and final (Nov 30&Dec 2)
 - **Strong projects will be invited to continue after quarter for paper submission.**

- **Grading**

- Paper Presentation and Discussion: 35%
- Discussions and Participations: 15%
- Course Project: 50%
 - Team List: 5 pts
 - Proposal: 15 pts
 - Mid-term: 20 pts
 - Final: 60 pts
- **Presentations and Q&A are important.**

- **Course Discussion, and Office Hour**
 - **Discussion:** Piazza. Link in the course website.
 - Access Code: **2wi54fzukmv**
 - **Office hour:** 4-5 PM, Thursdays @ Engineering VI, in front of Office 295
 - Course Projects
 - Papers discussions
 - Or anything else related to the topic (but w/ a lower priority...)

We greatly appreciate the generous support from Google G!

- **Compute Resource**

- **Course-specific Resource:**

- Google TPUs
 - TPU Technical Support from Google MaxText
 - Claude Pro Subscription

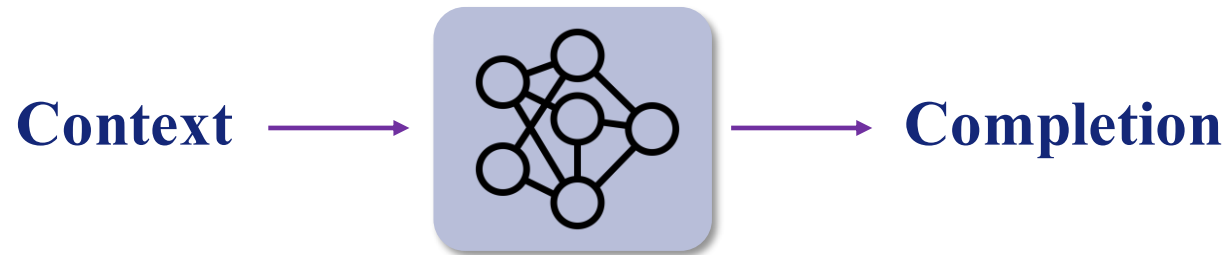
- **Other Resources**

- Free access to frontier agent through Amazon Kiro
 - Free access to GPUs through Colab/Google AI Student Plan
 - Paid resources or your own devices

- **PTEs:** Compute resources are the bottleneck
 - Students on or not on the waitlist:
 - Carefully check the paper list and **sign up for the presentation first**
 - Send me email w/ paper information
 - If you could not sign up for a paper due to the full capacity...
 - Find the classmate who is willing to share the presentation w/ you
 - Send me email w/ your co-presenter cc'ed
 - Approval is not guaranteed, but we will try

What is a Language Model?

$$P(\textit{Completion} \mid \textit{Context})$$



for (loop_ctr = 0; loop_ctr < 10;

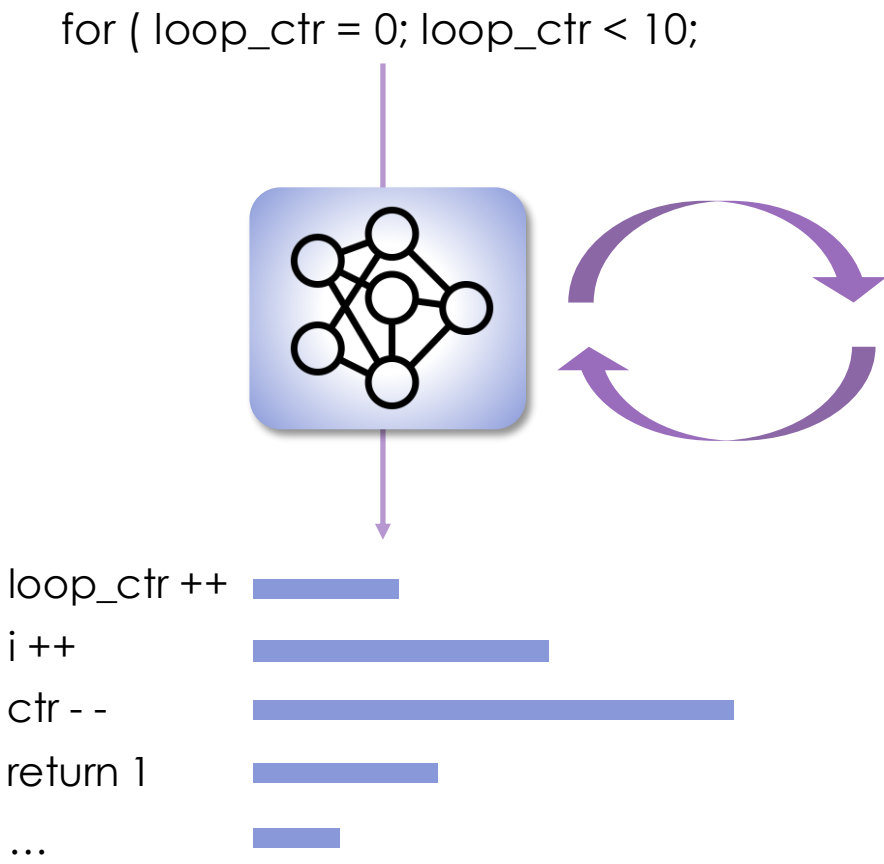
loop_ctr ++){

“Autoregressive”

for (____ = 0; loop_ctr < 10; loop_ctr ++){

loop_ctr

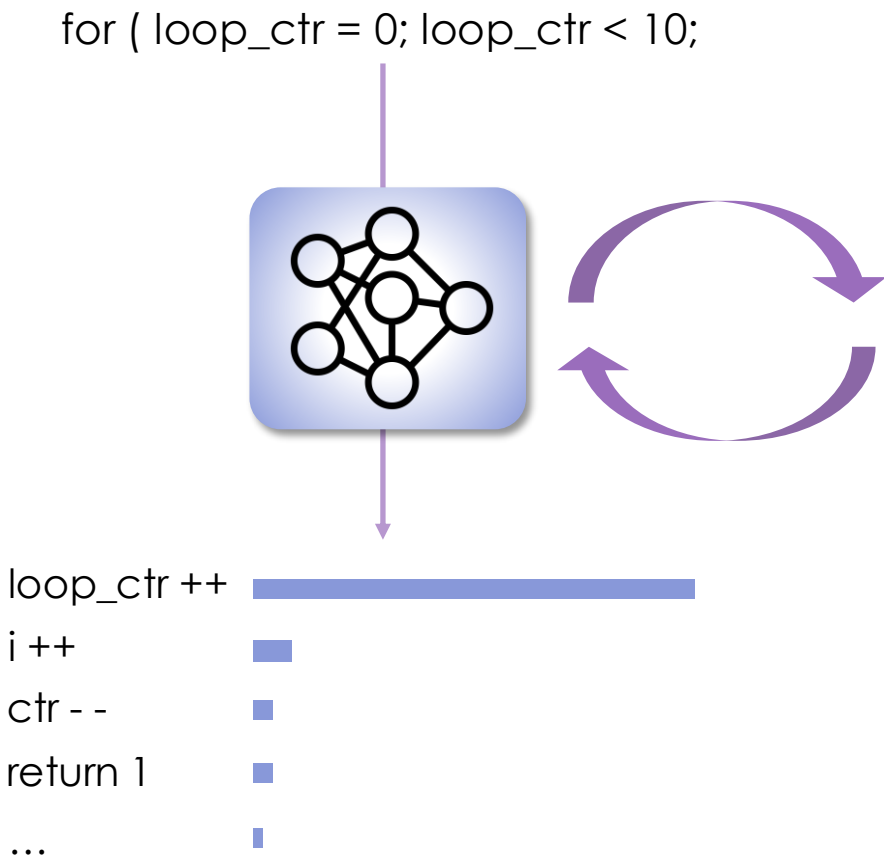
“Bi-Directional”



```
/*
Inference for Llama-2 Transformer model in pure C
*/
#include <stdio.h>
#include <stdlib.h>
...
// Transformer model
typedef struct {
    Config config; // the hyperparameters of the architecture
    TransformerWeights weights; // the weights of the model
    RunState state; // buffers for the "wave" of activations
    // some more state needed to properly clean up the memory
    int fd; // file descriptor for memory mapping
    float* data; // memory mapped data pointer
    ssize_t file_size; // size of the checkpoint file in bytes
} Transformer;

void build_transformer(Transformer* t, char* checkpoint_path) {
    // read in the Config and the Weights from the checkpoint
    read_checkpoint(checkpoint_path, &t->config, &t->weights, &t->fd, &t->data, &t->file_size);
    // allocate the RunState buffers
    malloc_run_state(&t->state, &t->config);
}
...
```

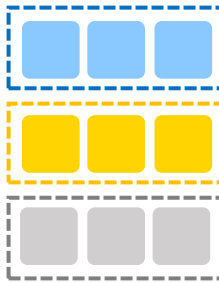




```
/*  
 * Inference for Llama-2 Transformer model in pure C  
 */  
#include <stdio.h>  
#include <stdlib.h>  
...  
// Transformer model  
typedef struct {  
    Config config; // the hyperparameters of the architecture  
    TransformerWeights weights; // the weights of the model  
    RunState state; // buffers for the "wave" of activations  
    // some more state needed to properly clean up the memory  
    int fd; // file descriptor for memory mapping  
    float* data; // memory mapped data pointer  
    ssize_t file_size; // size of the checkpoint file in bytes  
} Transformer;  
void build_transformer(Transformer* t, char* checkpoint_path) {  
    // read in the Config and the Weights from the checkpoint  
    read_checkpoint(checkpoint_path, &t->config, &t->weights, &t->fd, &t->data, &t->file_size);  
    // allocate the RunState buffers  
    malloc_run_state(&t->state, &t->config);  
} ...
```



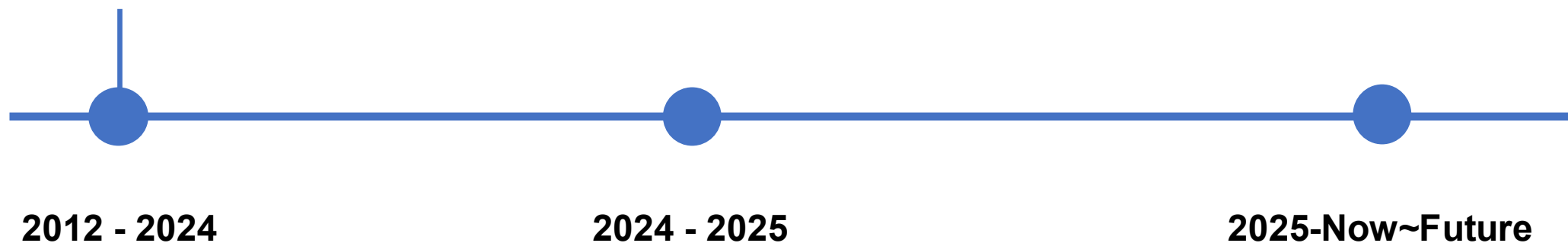
↓
Tokenized Code Files



Next Token Prediction

LLMs and Coding: The Brief History

LLM/AI advances coding



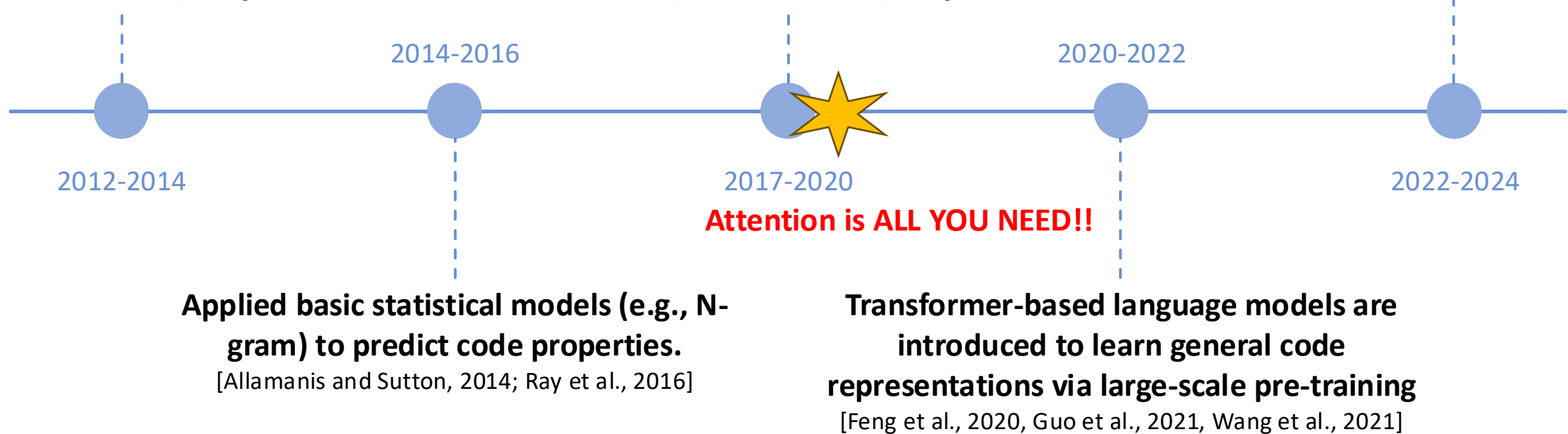
Observed the repetitiveness and statistical predictability of the software.

[Hindle et al., 2012; Allamanis et al., 2013; Barr et al., 2014]

Advanced statistical models (e.g., RNN) are adapted to code modeling.

[Yin and Neubig, 2017; Zhou et al., 2019; Hellendoorn et al., 2020]

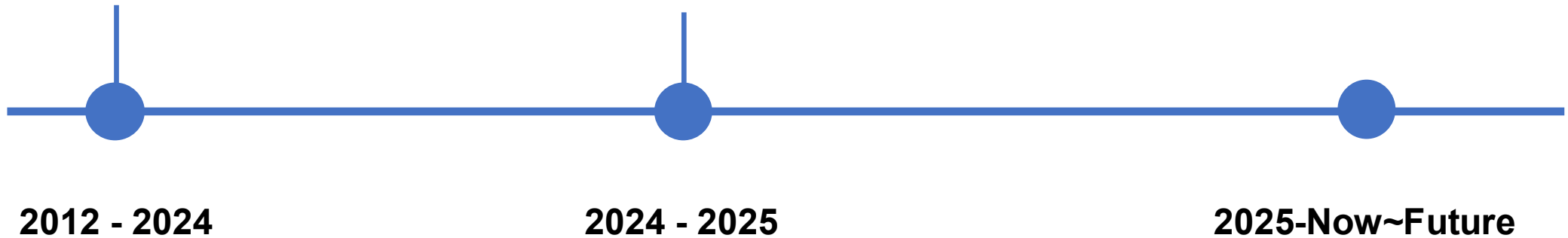
Large Language Models (LLM) w/ xB params are pre-trained w/ nT tokens. Products Deployed.



LLMs and Code Intelligence: The Brief History

LLM/AI advances coding

Code advances intelligence of LLM

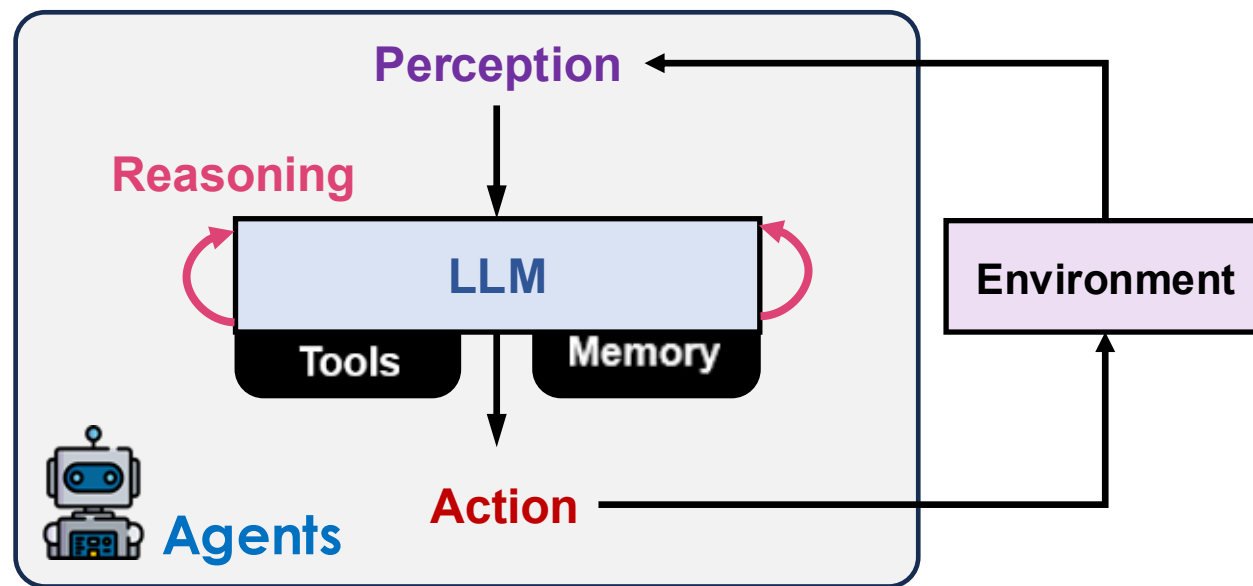


LLMs cannot reason about everything, but they don't have to.

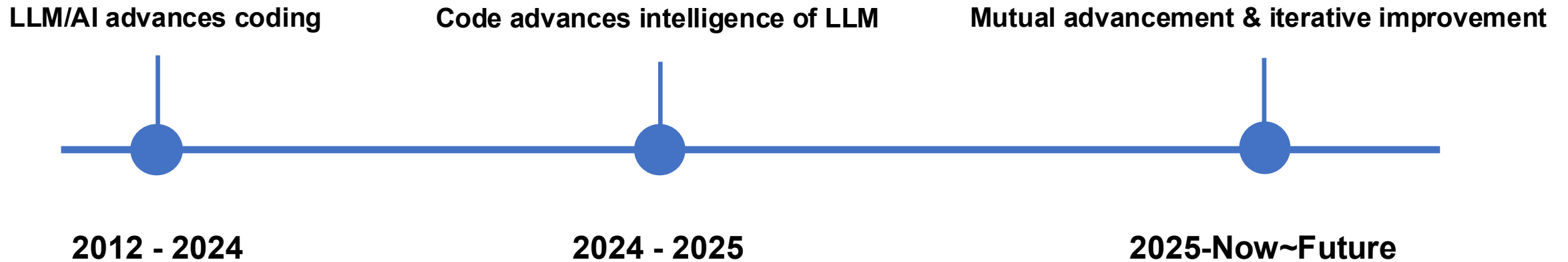


LLMs cannot verify by themselves, but w/ code, they can.

“LLM is a brain in the jar – it memorizes the facts but never has experiences”



LLMs and Code Intelligence: The Brief History



What is Code Intelligence?



Solve harder problems



*Realize better intelligence
& Self-Improvement*



- **Paper List Walk-through**



Introduce yourself & Find your teammate